

Diseño de una Línea de Productos de Software para Sistemas de Gestión de Restaurantes

Johanna Puerchambud, Hugo Pepinosa, Luis Flores.

Universidad Técnica del Norte

lhfloresc@utn.edu.ec

hapepinosam@utn.edu.ec

jepuerchambudp@utn.edu.ec

Abstract— This report proposes the design of a Software Product Line for restaurant management systems. The selected domain comprises platforms geared towards small restaurants, cafes, online ordering services, and restaurant chains. The model was built using the FeatureIDE tool to represent common features, variability, constraints between features, and valid product configurations. The proposal includes a communality and variability analysis, a Feature Model, a reference architecture, three valid configurations, a Configuration Knowledge table, and a return on investment (ROI) analysis. The results show that the Software Product Line approach allows for the reuse of common functionalities, reduces development costs, and facilitates the systematic derivation of new products.

KEYWORDS: Línea de Productos de Software, Feature Model, FeatureIDE, variabilidad, restaurantes, Configuration Knowledge.

I. INTRODUCCIÓN Y DOMINIO ELEGIDO

Las Líneas de Productos de Software permiten construir una familia de sistemas relacionados a partir de una base común de activos reutilizables. En lugar de desarrollar cada sistema desde cero, una LPS identifica las características compartidas por todos los productos y las características variables que permiten adaptar cada producto a un contexto específico.

En este trabajo se diseña una LPS para sistemas de gestión de restaurantes. Este dominio fue seleccionado porque existen varios productos relacionados que comparten funcionalidades comunes, como autenticación, gestión de menú, procesamiento de pedidos, pagos y reportes. Sin embargo, también presentan variaciones importantes, como gestión de mesas, reservas, pedidos en línea, delivery, promociones, inventario y administración de múltiples sucursales.

El objetivo principal es diseñar un modelo de variabilidad que permita derivar diferentes productos de software de forma controlada, reutilizando componentes comunes y aplicando restricciones que eviten configuraciones inválidas.

II. PORTAFOLIO DE PRODUCTOS

El dominio seleccionado corresponde a una Línea de Productos de Software para sistemas de gestión de restaurantes. Este dominio incluye aplicaciones orientadas a la administración de restaurantes tradicionales, cafeterías, servicios de delivery y cadenas de restaurantes.

Se definieron cuatro productos dentro de la línea:

Tabla I. Portafolio de productos de la LPS.

Producto	Descripción	Mercado objetivo
RestaurantBasic	Sistema para restaurantes pequeños o tradicionales con funcionalidades esenciales, gestión de mesas y reservas.	Restaurante s pequeños o medianos
CafePOS	Sistema ligero tipo punto de venta para cafeterías, orientado a ventas rápidas y control básico.	Cafeterías y locales de comida rápida

RestaurantDelivery	Sistema orientado a pedidos en línea y entregas a domicilio.	Restaurante s con servicio delivery
RestaurantChain	Sistema para cadenas de restaurantes con múltiples sucursales e inventario centralizado.	Cadenas y franquicias

La selección de este dominio se justifica porque todos los productos pertenecen a una misma familia de sistemas y comparten un núcleo funcional común. Al mismo tiempo, cada producto necesita características específicas según el tipo de negocio, lo cual permite aplicar adecuadamente el enfoque de LPS.

III. OBJETIVOS

Objetivo general

Diseñar una Línea de Productos de Software para sistemas de gestión de restaurantes mediante un Feature Model que represente características comunes, puntos de variación, restricciones y configuraciones válidas.

Objetivos específicos

- Identificar las características comunes y variables del dominio.
- Construir un Feature Model utilizando FeatureIDE.
- Definir restricciones cross-tree para controlar configuraciones inválidas.
- Diseñar una arquitectura de referencia para la familia de productos.
- Crear configuraciones válidas para productos específicos.
- Documentar el Configuration Knowledge del modelo.
- Realizar un análisis económico de retorno de inversión.

IV. JUSTIFICACIÓN DEL DOMINIO

El sector de restaurantes requiere soluciones tecnológicas adaptadas a distintos tipos de negocios. Un restaurante pequeño no necesita las mismas funciones que una cadena con varias sucursales, y una cafetería tiene necesidades distintas a un restaurante enfocado en delivery.

Por esta razón, el dominio es adecuado para aplicar una Línea de Productos de Software. Todos los productos comparten una base común, pero se diferencian por sus características variables. El enfoque LPS permite reutilizar componentes, reducir tiempos de desarrollo y mantener una arquitectura común.

Además, la variabilidad del dominio es clara.

Existen funcionalidades obligatorias, como autenticación, menú, pedidos, pagos y reportes. También existen funcionalidades opcionales o variables, como reservas, delivery, promociones, pedidos en línea, inventario y multi-sucursal.

V. ANÁLISIS DE COMMONALITY / VARIABILITY

El análisis de comunalidad y variabilidad permite identificar qué características están presentes en todos los productos y cuáles dependen del tipo de sistema que se desea derivar.

Tabla II. Análisis de comunalidad y variabilidad.

Feature	Tipo	Descripción	Productos que la incluyen
Authentication	Commonality	Permite el inicio de sesión seguro de usuarios y administradores.	Todos
MenuManagement	Commonality	Permite gestionar productos, precios, categorías y disponibilidad del menú.	Todos
OrderProcessing	Commonality	Permite registrar y procesar pedidos.	Todos
Payment	Commonality	Permite registrar pagos realizados	Todos

		por los clientes.	
Reports	Commodity	Permite generar reportes básicos de ventas y operaciones.	Todos
TableManagement	Variability	Permite administrar mesas dentro del restaurante.	RestaurantBasic, CafePOS
Reservations	Variability	Permite registrar reservas de clientes.	RestaurantBasic
Delivery	Variability	Permite gestionar entregas a domicilio.	RestaurantDelivery, RestaurantChain
OnlineOrdering	Variability	Permite recibir pedidos desde una plataforma en línea.	RestaurantDelivery, RestaurantChain
Inventory	Variability	Permite controlar existencias de productos e insumos.	RestaurantChain
MultiBranch	Variability	Permite administrar varias sucursales desde una misma plataforma.	RestaurantChain

Promotions	Variability	Permite gestionar promociones y descuentos.	CafePOS, RestaurantDelivery, RestaurantChain
Notifications	Variability	Permite enviar notificaciones a clientes o administradores.	RestaurantDelivery, RestaurantChain

Las características comunes forman el núcleo obligatorio del sistema. Las características variables permiten adaptar cada producto al mercado objetivo correspondiente.

VI. MODELADO DE CARACTERÍSTICAS (FEATURE MODELING)

Para el modelado de la Línea de Productos de Software se utilizó FeatureIDE, una herramienta que permite construir modelos de características, definir restricciones y validar configuraciones.

El proceso aplicado se dividió en tres etapas:

1. **Domain Scoping:** se seleccionó el dominio de sistemas de gestión de restaurantes y se definió el portafolio de productos.
2. **Domain Modeling:** se identificaron características comunes, variables y restricciones para construir el Feature Model.
3. **Domain Realization:** se definieron configuraciones de productos y se documentó el Configuration Knowledge que relaciona features con artefactos de implementación.

Este proceso permite organizar la variabilidad del dominio y establecer una base reutilizable para derivar productos concretos.

VII. FEATURE MODEL PROPUESTO

El Feature Model tiene como raíz la característica **RestaurantManagementSystem**. A partir de esta raíz se organizan los módulos principales del sistema: Core, Management, Services, Sales y ProductType.

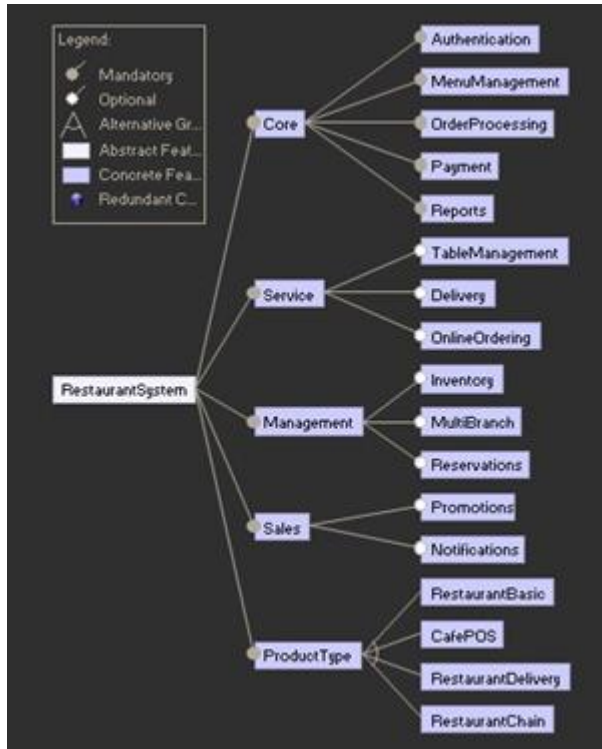


Fig. 1. Estructura lógica del Feature Model propuesto.

El grupo **Core** es obligatorio y representa las características que deben estar presentes en todos los productos. Los módulos **Management**, **Services** y **Sales** contienen características variables. El grupo **ProductType** es alternativo, por lo que cada configuración debe seleccionar exactamente un tipo de producto.

VIII. MODELO EN GUIDSL PARA FEATUREIDE

El siguiente modelo puede ser utilizado en FeatureIDE como representación textual del Feature Model:

```
RestaurantManagementSystem : Core [Management]
[Services] [Sales] ProductType ;

Core : Authentication MenuManagement
OrderProcessing Payment Reports ;

Management : [TableManagement] [Reservations]
[Inventory] [MultiBranch] ;

Services : [Delivery] [OnlineOrdering] ;

Sales : [Promotions] [Notifications] ;

ProductType : RestaurantBasic | CafePOS |
RestaurantDelivery | RestaurantChain ;
```

IX. PUNTOS DE VARIACIÓN Y MECANISMOS DE ENLACE

Los puntos de variación representan las decisiones del modelo donde los productos pueden diferir. Cada punto de variación contiene variantes que pueden ser seleccionadas o excluidas dependiendo del producto.

Tabla III. Puntos de variación de la línea.

Punto de Variación	Variantes	Mecanismo	Binding Time
ProductType	FastFood, FineDining, Delivery, Coffee	Feature Model	Compile-time
Service	Delivery, OnlineOrdering	Feature Flags	Runtime
Management	Inventory, MultiBranch	Dependency Injection	Deployment
Sales	Promotions, Notifications	Strategy Pattern	Runtime

La elección de los mecanismos de enlace responde a las características de cada grupo de variabilidad. El ProductType se resuelve en tiempo de compilación porque define la identidad del producto y no debe cambiar durante la ejecución. Las características del módulo Services, como Delivery y OnlineOrdering, utilizan Feature Flags en tiempo de ejecución para permitir activación dinámica sin recompilación. Las características de Management, como Inventory y MultiBranch, utilizan inyección de dependencias en tiempo de despliegue, ya que pueden alterar la estructura de la base de datos o requerir configuración adicional. Finalmente, el módulo Sales utiliza el patrón Strategy para intercambiar comportamientos como promociones o notificaciones sin afectar el resto del sistema.

X. CROSS-TREE CONSTRAINTS

Las restricciones cross-tree permiten controlar dependencias y exclusiones entre características ubicadas en diferentes ramas del modelo.

Tabla IV. Restricciones cross-tree del modelo.

Restricción	Justificación
RestaurantBasic $\Rightarrow$ TableManagement	Un restaurante tradicional necesita controlar mesas.
RestaurantBasic $\Rightarrow$ Reservations	Un restaurante tradicional puede requerir reservas.

CafePOS $\Rightarrow$ TableManagement	Una cafetería puede requerir gestión básica de mesas o puntos de atención.
RestaurantDelivery $\Rightarrow$ Delivery $\wedge$ OnlineOrdering	Un sistema delivery necesita pedidos en línea y entregas.
RestaurantChain $\Rightarrow$ MultiBranch $\wedge$ Inventory	Una cadena de restaurantes requiere múltiples sucursales e inventario.
OnlineOrdering $\Rightarrow$ Payment	Los pedidos en línea requieren pagos asociados.
Delivery $\Rightarrow$ Notifications	Las entregas requieren notificar estados del pedido.
MultiBranch $\Rightarrow$ Inventory	La gestión multi-sucursal necesita control de inventario.
RestaurantBasic $\Rightarrow$ $\neg$ MultiBranch	Un restaurante básico no administra varias sucursales.
CafePOS $\Rightarrow$ $\neg$ Delivery	El sistema de cafetería POS se enfoca en atención presencial.

Estas restricciones evitan configuraciones incorrectas, como seleccionar RestaurantDelivery sin OnlineOrdering o RestaurantChain sin MultiBranch.

XI. CONFIGURACIONES DE PRODUCTOS

A. Configuración 1: RestaurantDelivery

Tabla V. Configuración RestaurantDelivery.

Característica	Valor
Core: Authentication, MenuManagement, OrderProcessing, Payment, Reports	Seleccionado, obligatorio
Delivery	Seleccionado
OnlineOrdering	Seleccionado
Notifications	Seleccionado
ProductType	RestaurantDelivery
Estado	Válida

La configuración RestaurantDelivery incluye todas las características obligatorias del núcleo y agrega funcionalidades propias de pedidos en línea y entregas a domicilio. Esta configuración es válida porque cumple la restricción $\text{RestaurantDelivery} \Rightarrow \text{Delivery} \wedge \text{OnlineOrdering}$. Además, Delivery activa Notifications, lo que permite informar al cliente sobre el estado del pedido.

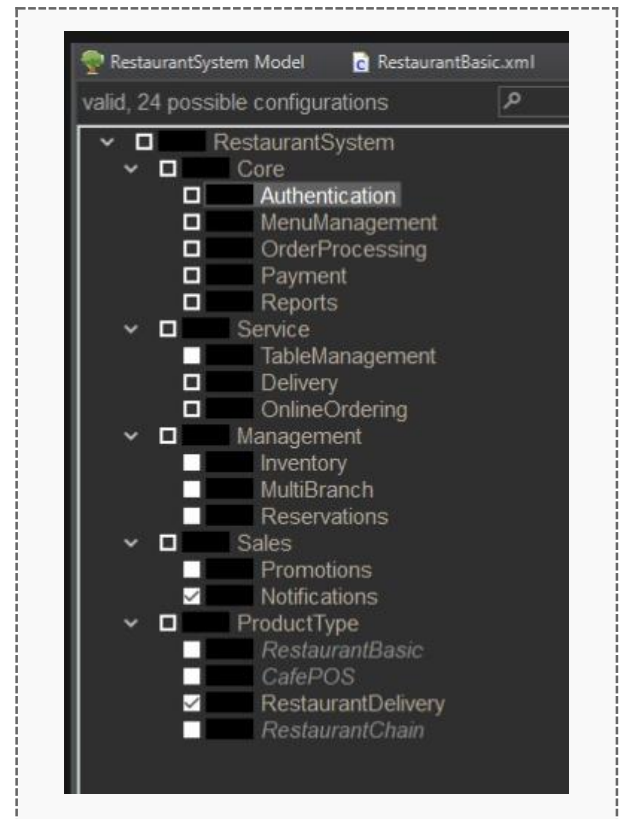


Fig. 4. Validación de RestaurantDelivery en FeatureIDE.

B. Configuración 2: RestaurantBasic

Tabla VI. Configuración RestaurantBasic.

Característica	Valor
Core: Authentication, MenuManagement, OrderProcessing, Payment, Reports	Seleccionado, obligatorio
TableManagement	Seleccionado
Reservations	Seleccionado
ProductType	RestaurantBasic
Estado	Válida

La configuración RestaurantBasic incluye todas las características obligatorias del núcleo, además de funcionalidades como gestión de mesas y reservas. Esto es consistente con un restaurante tradicional que atiende clientes presencialmente y puede requerir administración de reservas. La configuración es válida ya que cumple todas las restricciones del modelo.

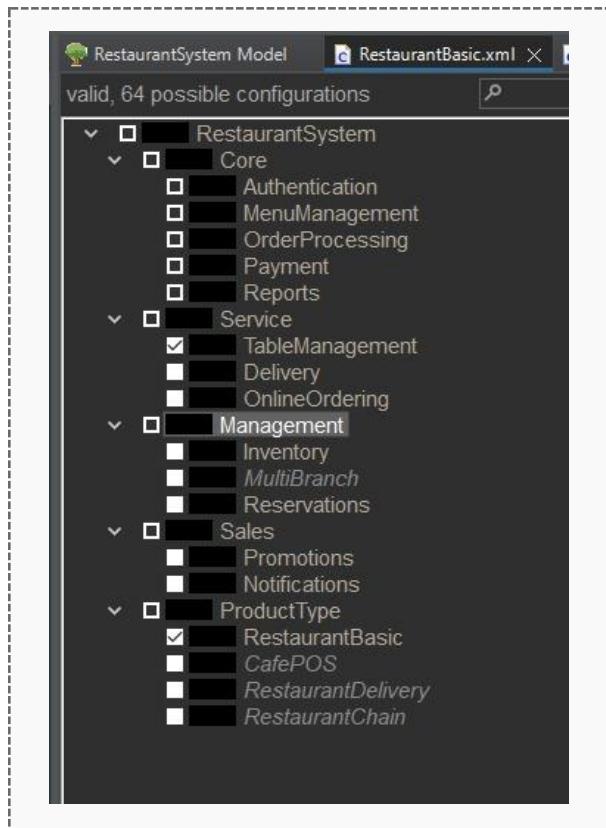


Fig. 5. Validación de RestaurantBasic en FeatureIDE.

C. Configuración 3: RestaurantChain

Tabla VII. Configuración RestaurantChain.

Característica	Valor
Core: Authentication, MenuManagement, OrderProcessing, Payment, Reports	Seleccionado, obligatorio
Inventory	Seleccionado
MultiBranch	Seleccionado
Delivery	Seleccionado
OnlineOrdering	Seleccionado
Notifications	Seleccionado
Promotions	Seleccionado
ProductType	RestaurantChain
Estado	Válida

La configuración RestaurantChain representa una cadena de restaurantes con varias sucursales. Incluye MultiBranch e Inventory porque una cadena necesita administrar diferentes locales y controlar existencias. También puede incluir Delivery y OnlineOrdering para ofrecer pedidos en línea, además de Notifications y Promotions para mejorar la comunicación con clientes y gestionar campañas comerciales.

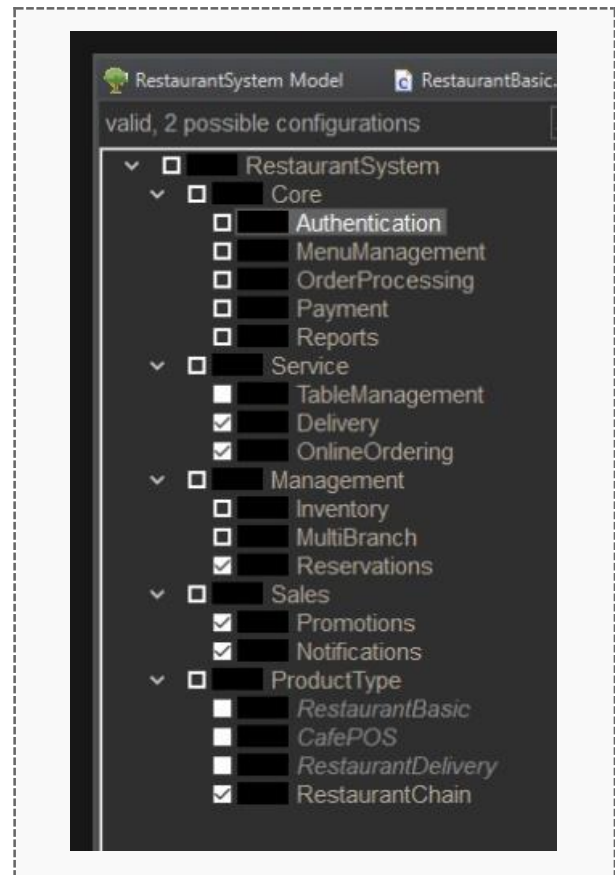


Fig. 6. Validación de RestaurantChain en FeatureIDE.

XII. VALIDACIÓN LÓGICA DE CONFIGURACIONES

A. Validación de RestaurantDelivery

Se verifica que la selección de la feature RestaurantDelivery active obligatoriamente las características Delivery y OnlineOrdering, conforme a la restricción $\text{RestaurantDelivery} \Rightarrow \text{Delivery} \wedge \text{OnlineOrdering}$. Además, la característica Delivery activa Notifications, permitiendo notificar al cliente sobre el estado del pedido. La configuración resulta válida porque incluye todas las características obligatorias del Core y cumple las restricciones definidas.

B. Validación de RestaurantBasic

Se verifica que la selección de RestaurantBasic sea compatible con características de atención presencial, como TableManagement y Reservations. Además, esta configuración no incluye MultiBranch, cumpliendo la restricción $\text{RestaurantBasic} \Rightarrow \neg \text{MultiBranch}$. La configuración resulta válida porque incluye el Core completo: Authentication, MenuManagement, OrderProcessing, Payment y Reports.

C. Validación de RestaurantChain

Se verifica que la selección de la feature RestaurantChain active obligatoriamente las características MultiBranch e Inventory, conforme a la restricción RestaurantChain $\Rightarrow$ MultiBranch $\wedge$ Inventory. Adicionalmente, la configuración puede incluir funcionalidades como Delivery y OnlineOrdering, respetando las dependencias definidas en el modelo. La validación en FeatureIDE confirma que la configuración es válida y no presenta conflictos.

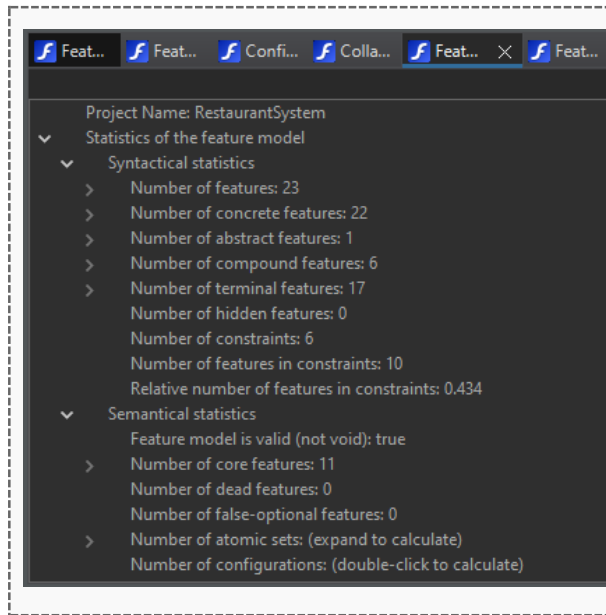


Fig. 7. Estadísticas y validación general del modelo.

XIII. CONFIGURATION KNOWLEDGE

El Configuration Knowledge permite relacionar cada característica del modelo con los artefactos de implementación que deben incluirse, excluirse o modificarse al derivar un producto.

Tabla VIII. Configuration Knowledge.

Feature	Artefacto	Acción
Authentication	/src/auth/, auth.controller.ts, jwt.service.ts	Inclusión Core: obligatorio en todos los productos
MenuManagement	/src/menu/, menu.controller.ts, menu.service.ts	Inclusión Core: obligatorio en todos los productos
OrderProcessing	/src/orders/, order.controller.ts, order.service.ts	Inclusión Core: obligatorio en todos los productos
Payment	/src/payment/, payment.service.ts, payment.controller.ts	Inclusión Core: obligatorio y requerido por OnlineOrdering

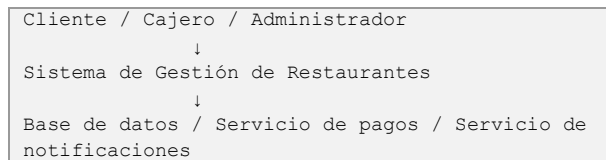
Reports	/src/reports/, report.service.ts, report.controller.ts	Inclusión Core: obligatorio en todos los productos
TableManagement	/src/tables/, table.service.ts	Inclusión opcional: activo en RestaurantBasic y CafePOS
Reservations	/src/reservations/, reservation.service.ts	Inclusión opcional: activo en RestaurantBasic
Delivery	/src/delivery/, delivery.service.ts	Inclusión opcional: activo en RestaurantDelivery y RestaurantChain
OnlineOrdering	/src/online-orders/, order-api.ts	Inclusión opcional: activo en RestaurantDelivery y RestaurantChain
Inventory	/src/inventory/, inventory.service.ts	Inclusión opcional: activo en RestaurantChain
MultiBranch	/src/branches/, branch.service.ts	Inclusión opcional: activo en RestaurantChain
Promotions	/src/promotions/, promo.service.ts	Inclusión opcional: activo en CafePOS, RestaurantDelivery y RestaurantChain
Notifications	/src/notifications/, notification.service.ts	Inclusión opcional: requerido por Delivery

Esta tabla permite mantener trazabilidad entre las decisiones del Feature Model y los componentes de implementación. Por ejemplo, si se deriva el producto RestaurantDelivery, se incluyen los módulos Delivery, OnlineOrdering y Notifications. En cambio, si se deriva RestaurantBasic, no se incluyen módulos como MultiBranch o Inventory.

XIV. ARQUITECTURA DE REFERENCIA

La arquitectura de referencia define los componentes principales que serán reutilizados por todos los productos de la línea. Esta arquitectura se organiza en capas para separar responsabilidades y facilitar la variabilidad.

A. C4 Nivel 1 — Contexto



El sistema interactúa con diferentes tipos de usuarios: clientes, cajeros, administradores de restaurante y administradores de sucursal. También puede integrarse con servicios externos, como pasarelas de pago y servicios de notificaciones.

B. C4 Nivel 2 — Contenedores

Tabla IX. Contenedores de la arquitectura de referencia.

Contenedor	Responsabilidad
Web Admin Panel	Permite administrar menú, usuarios, reportes, inventario y sucursales.
POS Interface	Permite registrar pedidos y pagos en restaurante o cafetería.
Online Ordering App	Permite a clientes realizar pedidos en línea.
Backend API	Gestiona autenticación, menú, pedidos, pagos, reportes y reglas de negocio.
Notification Service	Envía notificaciones de pedidos, entregas y promociones.
Payment Service	Gestiona pagos presenciales o en línea.
Database	Almacena usuarios, productos, pedidos, pagos, reportes, sucursales e inventario.

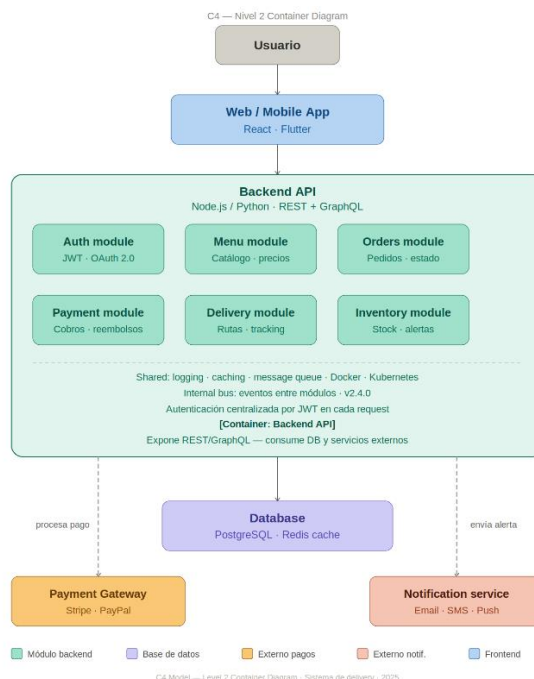


Fig. 8. Arquitectura de referencia de la LPS.

C. Relación con puntos de variación

La arquitectura permite activar o desactivar módulos dependiendo de la configuración seleccionada. Por ejemplo, RestaurantBasic puede utilizar POS Interface, Backend API y Database, mientras que RestaurantDelivery activa además Online Ordering App y Notification Service. RestaurantChain activa módulos adicionales como Inventory y MultiBranch.

XIII. REPOSITORIO DE IMPLEMENTACIÓN Y CONTROL DE VERSIONES

El desarrollo técnico, los artefactos de código y los modelos de configuración de la LPS se encuentran alojados y documentados en un repositorio público de GitHub. Este espacio sirve como el sistema central de gestión de activos base (*Core Assets*) y facilita la integración continua del proyecto.

A. Estructura del Repositorio

El repositorio está organizado siguiendo los principios de separación de intereses de una LPS, permitiendo el acceso independiente a la lógica de negocio y a los modelos de variabilidad:

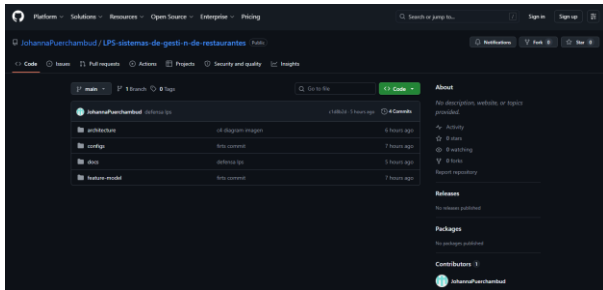
- **Modelos de Características:** Archivos .xml y configuraciones específicas desarrolladas en FeatureIDE.

- **Código Fuente:** Implementación modular de los componentes *Core* (autenticación, pedidos) y variantes (delivery, inventario).
- **Documentación Técnica:** Guías para la derivación de productos y especificaciones de la arquitectura de referencia.

B. Acceso al Proyecto

Para fines de revisión técnica, auditoría de código y replicabilidad del experimento, el proyecto está disponible en la siguiente dirección:

URL del Proyecto:
<https://github.com/JohannaPuerchambud/LPS-sistemas-de-gesti-n-de-restaurantes.git>



XIII. ANÁLISIS ECONÓMICO Y DE RETORNO DE INVERSIÓN (ROI)

El éxito de una LPS se mide por su capacidad para amortizar la inversión inicial en activos base a través de la derivación de múltiples productos. Para este análisis, se han definido las siguientes variables económicas:

A. Modelo Económico

Se utilizan los siguientes parámetros de costo:

$\$C_{DE}$ (Costo de Desarrollo de la Línea): **9000 USD** (Inversión inicial en arquitectura, Core Assets y herramientas).

C_{AE} (Costo de Desarrollo Ad-hoc): **4000 USD** (Costo estimado de construir un sistema único desde cero).

Costo de Derivación con LPS: **1500 USD** (Costo de configurar y desplegar un nuevo producto basado en la línea).

B. Punto de Equilibrio (Break-even Point)

El ahorro por cada producto derivado se calcula como la diferencia entre el desarrollo tradicional y el costo de derivación:

$$\text{Ahorro} = C_{AE} - \text{Costo}_{LPS} = 4000 - 1500 = 2500 \text{ USD}$$

Para hallar el punto de equilibrio (n), dividimos el costo de la línea por el ahorro por producto:

$$n = C_{DE} / \text{Ahorro} = 9000 / 2500 = 3.6$$

Resultado: El punto de equilibrio se alcanza en el producto 4. A partir de esta entrega, la empresa comienza a generar utilidades netas superiores a las que obtendría mediante el desarrollo independiente (ad-hoc).

Como lección aprendida, se evidencia que el enfoque de Línea de Productos de Software requiere una inversión inicial mayor en análisis, arquitectura y modelado de variabilidad. Sin embargo, esta inversión se justifica cuando se desarrollan varios productos relacionados, ya que permite reutilizar componentes comunes y reducir el esfuerzo de derivación. En un trabajo futuro se podría mejorar el modelo incorporando métricas reales de desarrollo, integración con un repositorio funcional y generación automática de productos desde FeatureIDE.

XV. CONCLUSIONES Y LECCIONES APRENDIDAS

A. Hallazgos Principales

Escalabilidad: El enfoque LPS permite una respuesta rápida a las demandas del mercado gastronómico, logrando una reducción del tiempo de salida al mercado (time-to-market) tras el desarrollo del núcleo.

Calidad: La reutilización de componentes core validados disminuye la incidencia de errores en las versiones finales de los productos específicos.

B. Qué se haría diferente

En una iteración futura, se recomendaría la implementación de un sistema de Pruebas Automatizadas para Variabilidad, asegurando que cada posible combinación de características en FeatureIDE pase un control de calidad automático antes de la derivación. Además, se exploraría un tiempo de enlace (binding time) dinámico para permitir actualizaciones de módulos en tiempo de ejecución sin reiniciar los servicios.

XVI. BIBLIOGRAFÍA.



- [1] K. Pohl, G. Böckle, and F. J. van der Linden, *Software Product Line Engineering: Foundations, Principles and Techniques*. Berlin, Germany: Springer-Verlag, 2005.
- [2] P. Clements and L. Northrop, *Software Product Lines: Practices and Patterns*. Boston, MA: Addison-Wesley, 2002.
- [3] T. Thüm, C. Kästner, F. Benduhn, J. Meinicke, G. Saake, and T. Leich, "FeatureIDE: An extensible framework for feature-oriented software development," *Sci. Comput. Program.*, vol. 79, pp. 70–85, Jan. 2014. [Online]. Available: <https://doi.org/10.1016/j.scico.2012.06.002>
- [4] S. Apel, D. Batory, C. Kästner, and G. Saake, *Feature-Oriented Software Product Lines*. Berlin, Germany: Springer, 2013.
- [5] C. Simons, "The C4 model for visualizing software architecture," 2025. [Online]. Available: <https://c4model.com/>
- [6] J. Puerchambud, "LPS-sistemas-de-gestión-de-restaurantes," GitHub Repository, 2026. [Online]. Available: <https://github.com/JohannaPuerchambud/LPS-sistemas-de-gesti-n-de-restaurantes>